UNIVERSIDAD NACIONAL DE UCAYALI

FACULTAD DE INGENIERÍA DE SISTEMAS E INGENIERÍA CIVIL

ESCUELA DE PROFESIONAL DE INGENIERÍA DE SISTEMAS

PROGRAMACIÓN ORIENTADA A OBJETOS II

Docente : Ing. M.Sc. Edson Greig Cáceres Cárdenas

Pucallpa – Perú

2026

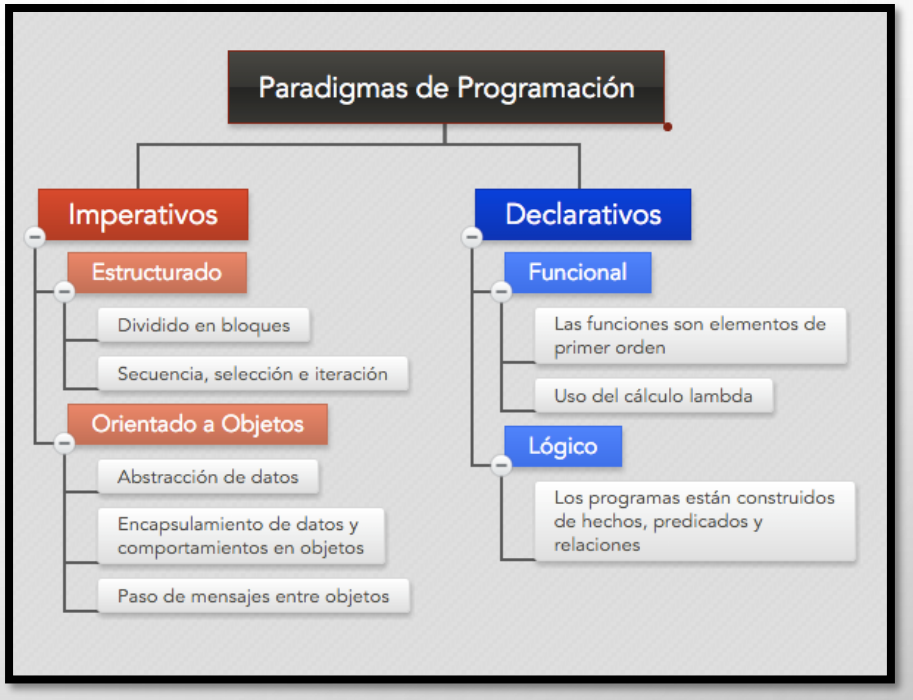
TABLA DE CONTENIDO

- CAPÍTULO I: ¿Qué es un paradigma de programación?
- CAPÍTULO II: Programación orientada a objetos (POO)
- CAPÍTULO III: Pilares de la POO
- CAPÍTULO IV: Ventajas y desventajas de la POO
- CAPÍTULO V: Conclusión

CAPÍTULO I: ¿QUÉ ES UN PARADIGMA DE PROGRAMACIÓN?

Un **paradigma de programación** es, en esencia, una filosofía o un estilo de programar. No se trata de un lenguaje específico (como Python o Java), sino del **enfoque o la mentalidad** que sigues para resolver un problema y estructurar el código.

Un **paradigma de programación** es una forma relativamente avanzada de conceptualizar y estructurar la implementación de un programa informático.



CAPÍTULO I: ¿QUÉ ES UN PARADIGMA DE PROGRAMACIÓN?

Descripción de la realidad problemática

Paradigma	Enfoque	Ventajas	Desventajas
Imperativo	Describe cómo debe cambiar el estado (paso a paso).	Fácil de entender para algoritmos simples; eficiente.	Difícil de mantener en proyectos grandes (código espagueti).
Funcional	Basado en funciones matemáticas y datos inmutables.	Menos errores (bugs), ideal para procesamiento paralelo.	Curva de aprendizaje alta; puede ser menos eficiente en memoria.
Orientado a Objetos	Organiza el código en "objetos" que contienen datos y lógica.	Reutilización de código, modularidad y fácil mantenimiento.	Mayor consumo de recursos; diseño inicial más complejo.

¿Por qué es importante conocerlos?

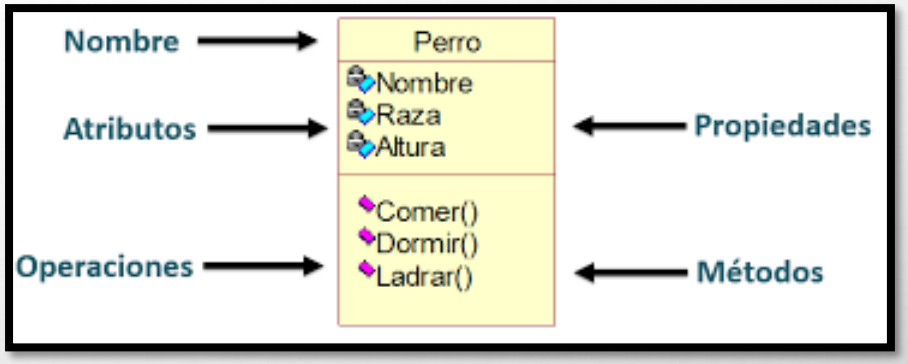
Entender los paradigmas te hace un programador más **versátil**. Si solo conoces la programación imperativa, intentarás resolver todo con bucles *for* infinitos, cuando quizás una función de orden superior o un objeto bien definido resolvería el problema en la mitad de líneas y con menos errores.

```python
# Función para aplicar una operación a todos los elementos de una lista
def apply_operation(nums, operation):
  return list(map(operation, nums))

# Uso de la función con una lambda para elevar al cuadrado cada elemento de la
lista
nums = [1, 2, 3, 4, 5]
squared_nums = apply_operation(nums, lambda x: x**2)
print(squared_nums) # imprime [1, 4, 9, 16, 25]
```

CAPÍTULO II: PROGRAMACIÓN ORIENTADA A OBJETOS

La **Programación Orientada a Objetos (POO)** busca modelar el mundo real a través de entidades digitales. En lugar de ver el programa como una lista de instrucciones, lo vemos como una red de objetos que se comunican entre sí.



Para entender la POO, debemos dominar estos dos términos:

1. **Clase**: Es el "molde" o plano. Define qué atributos y comportamientos tendrá un objeto. (Ejemplo: Un perro).
2. **Objeto**: Es la "instancia" o la creación física basada en la clase. (Ejemplo: Tu mascota llamada firulais).

CAPÍTULO II: PROGRAMACIÓN ORIENTADA A OBJETOS

```java
// Definición de la Clase
class Perro {
    String nombre;

    // Método (Acción)
    void ladrar() {
        System.out.println(nombre + " dice: ¡Guau guau!");
    }
}

// Clase principal para ejecutar el código
public class Main {
    public static void main(String[] args) {
        // Crear el objeto (Instanciación)
        Perro miPerro = new Perro();

        // Asignar un valor al atributo
        miPerro.nombre = "Toby";

        // Hacer que el objeto actúe
        miPerro.ladrar();
    }
}
```

CAPÍTULO III: PILARES DE LA POO

Estos son los fundamentos que sostienen este paradigma:

A. Abstracción

Consiste en captar las características esenciales de un objeto, ignorando los detalles irrelevantes.

- Ejemplo: Para manejar un coche, solo necesitas saber usar el volante y los pedales (interfaz), no necesitas entender cómo funciona la combustión interna del motor.

B. Encapsulamiento

Es la capacidad de ocultar los datos internos de un objeto y protegerlos de accesos no autorizados desde el exterior. Se logra mediante "modificadores de acceso" (público, privado, protegido).

- Ventaja: Seguridad y control sobre cómo se modifican los datos.

CAPÍTULO III: PILARES DE LA POO

C. Herencia

Permite que una clase nueva (clase hija) adquiera las propiedades y métodos de una clase existente (clase padre).

Ejemplo: Una clase Animal puede heredar sus rasgos a Perro y Gato. Ambos son animales, pero cada uno tiene sus particularidades.

D. Polimorfismo

Es la capacidad de que un mismo método se comporte de manera diferente según el objeto que lo llame.

Ejemplo: El método hacerSonido(). Si lo llama un Perro, hace "Guau"; si lo llama un Gato, hace "Miau".

CAPÍTULO IV: VENTAJAS Y DESVENTAJAS DE LA POO

Ventajas ✅

- Reutilización de código: Gracias a la herencia, no tienes que escribir lo mismo varias veces.
- Mantenibilidad: Si hay un error en un objeto, es más fácil aislarlo y arreglarlo sin romper todo el sistema.
- Escalabilidad: Ideal para proyectos grandes donde trabajan muchos desarrolladores simultáneamente.

Desventajas ❌

- Complejidad inicial: Requiere una planeación mucho más profunda antes de empezar a escribir código.
- Rendimiento: Los programas en POO suelen ser ligeramente más pesados y lentos que los procedimientos puros debido a la capa de abstracción.

CAPÍTULO V: CONCLUSION

La Programación Orientada a Objetos no es simplemente una técnica de escritura de código, sino un marco mental que ha dominado la industria del software durante más de tres décadas. Su éxito radica en que permite a los desarrolladores mapear problemas complejos del mundo real en estructuras lógicas manejables.

Algunas consideraciones:

1. **Dominio en la Industria:** Aunque lenguajes más modernos han ganado terreno, los pilares de la POO siguen siendo la base de los sistemas empresariales más grandes del mundo (banca, e-commerce, sistemas operativos) construidos en Java, C# o C++.
2. **La Era de la Programación Multiparadigma:** Es importante notar que hoy en día la mayoría de los lenguajes son "híbridos". Por ejemplo, algunos lenguajes permiten mezclar la POO con la Programación Funcional. Un buen desarrollador no se limita a un solo estilo, sino que usa la POO para la estructura y la Funcional para el procesamiento de datos.
3. **Más allá del Código:** Aprender POO entrena el pensamiento sistémico. Te enseña a pensar en términos de responsabilidades, interfaces y contratos, lo cual es esencial para entender arquitecturas modernas como los Microservicios.

GRACIAS